

1. Gemini and copilot
 2. access from the school lab computer
 3. prompt:
- Use the following requirements to generate C code:

The program will prioritize emails for a busy company CEO.

You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

You must create functions from scratch. Do not include pre-existing heap modules.

Test File format:

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

Boss – read first
Subordinate – read next
Peer – read next
ImportantPerson – read next
OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas.

Assume <date> is in the format: MM-DD-YYYY

The program must support the following commands:

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ

CEO has read email and dealt with it.

COUNT

Display current count of unread emails.

The program should be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), and two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Generate the complete C program.

4.gmini:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define INITIAL_CAPACITY 10
```

```
// Email structure
```

```
typedef struct {
```

```
    char category[50];
```

```
    int category_priority;
```

```
    char subject[256];
```

```
    char date[11];
```

```
    long date_val;
```

```
} Email;
```

```
// MaxHeap structure (List-based / Dynamic Array)
```

```
typedef struct {
```

```
    Email* data;
```

```
    int size;
```

```
    int capacity;
```

```
} MaxHeap;
```

```
// Function declarations
```

```

MaxHeap* create_heap();
void free_heap(MaxHeap* heap);
int get_category_priority(const char* cat);
long parse_date(const char* date_str);
int compare_emails(Email a, Email b);
void swap(Email* a, Email* b);
void sift_up(MaxHeap* heap, int index);
void sift_down(MaxHeap* heap, int index);
void insert_email(MaxHeap* heap, Email email);
Email extract_max(MaxHeap* heap);
Email peek_max(MaxHeap* heap);
void trim_whitespace(char* str);
void process_command(MaxHeap* heap, char* line);

```

```
// Main function
```

```
int main(int argc, char* argv[]) {
```

```
    FILE* file = stdin;
```

```
    if (argc > 1) {
```

```
        file = fopen(argv[1], "r");
```

```
        if (!file) {
```

```
            perror("Error opening file");
```

```
            return 1;
```

```
        }
```

```
    }
```

```
    MaxHeap* heap = create_heap();
```

```
    char line[512];
```

```
    while (fgets(line, sizeof(line), file)) {
```

```
        // Remove trailing newline/carriage return
```

```
        line[strcspn(line, "\r\n")] = 0;
```

```
        if (strlen(line) == 0) continue;
```

```
        process_command(heap, line);
```

```
    }
```

```
    if (file != stdin) {
```

```
        fclose(file);
```

```
    }
```

```
    free_heap(heap);
```

```
    return 0;
```

```
}
```

```

// Helper: Convert category to numerical priority
int get_category_priority(const char* cat) {
    if (strcmp(cat, "Boss") == 0) return 5;
    if (strcmp(cat, "Subordinate") == 0) return 4;
    if (strcmp(cat, "Peer") == 0) return 3;
    if (strcmp(cat, "ImportantPerson") == 0) return 2;
    if (strcmp(cat, "OtherPerson") == 0) return 1;
    return 0;
}

// Helper: Convert MM-DD-YYYY to a comparable integer YYYYMMDD
long parse_date(const char* date_str) {
    int mm, dd, yyyy;
    if (sscanf(date_str, "%d-%d-%d", &mm, &dd, &yyyy) == 3) {
        return (long)yyyy * 10000 + mm * 100 + dd;
    }
    return 0;
}

// Compare two emails: returns 1 if 'a' has higher priority than 'b', 0 otherwise.
// Higher category priority comes first. For the same category, newer date comes first.
int compare_emails(Email a, Email b) {
    if (a.category_priority != b.category_priority) {
        return a.category_priority > b.category_priority;
    }
    // Newer date means larger date value
    return a.date_val > b.date_val;
}

// Create heap
MaxHeap* create_heap() {
    MaxHeap* heap = (MaxHeap*)malloc(sizeof(MaxHeap));
    heap->capacity = INITIAL_CAPACITY;
    heap->size = 0;
    heap->data = (Email*)malloc(heap->capacity * sizeof(Email));
    return heap;
}

// Free heap memory
void free_heap(MaxHeap* heap) {
    if (heap) {
        free(heap->data);
        free(heap);
    }
}

```

```
}  
}
```

```
// Swap two emails
```

```
void swap(Email* a, Email* b) {  
    Email temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
// Sift Up operation
```

```
void sift_up(MaxHeap* heap, int index) {  
    while (index > 0) {  
        int parent = (index - 1) / 2;  
        if (compare_emails(heap->data[index], heap->data[parent])) {  
            swap(&heap->data[index], &heap->data[parent]);  
            index = parent;  
        } else {  
            break;  
        }  
    }  
}
```

```
// Sift Down operation
```

```
void sift_down(MaxHeap* heap, int index) {  
    int max_index = index;  
    int left = 2 * index + 1;  
    int right = 2 * index + 2;  
  
    if (left < heap->size && compare_emails(heap->data[left], heap->data[max_index])) {  
        max_index = left;  
    }  
    if (right < heap->size && compare_emails(heap->data[right], heap->data[max_index])) {  
        max_index = right;  
    }  
  
    if (max_index != index) {  
        swap(&heap->data[index], &heap->data[max_index]);  
        sift_down(heap, max_index);  
    }  
}
```

```
// Insert email into MaxHeap
```

```
void insert_email(MaxHeap* heap, Email email) {
```

```

    if (heap->size >= heap->capacity) {
        heap->capacity *= 2;
        heap->data = (Email*)realloc(heap->data, heap->capacity * sizeof(Email));
    }
    heap->data[heap->size] = email;
    sift_up(heap, heap->size);
    heap->size++;
}

```

```

// Extract max email (READ action)
Email extract_max(MaxHeap* heap) {
    Email empty = {"", 0, "", "", 0};
    if (heap->size == 0) return empty;

    Email root = heap->data[0];
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    sift_down(heap, 0);
    return root;
}

```

```

// Peek max email (NEXT action)
Email peek_max(MaxHeap* heap) {
    Email empty = {"", 0, "", "", 0};
    if (heap->size == 0) return empty;
    return heap->data[0];
}

```

```

// Trim leading and trailing whitespace
void trim_whitespace(char* str) {
    char* end;
    while(isspace((unsigned char)*str)) str++;
    if(*str == 0) return;
    end = str + strlen(str) - 1;
    while(end > str && isspace((unsigned char)*end)) end--;
    end[1] = '\0';
}

```

```

// Process individual commands from the test file/input
void process_command(MaxHeap* heap, char* line) {
    if (strncmp(line, "EMAIL", 5) == 0) {
        // Format: EMAIL <sender category>, <subject line>, <date>
        char* content = line + 5;
        while (*content && isspace((unsigned char)*content)) content++;
    }
}

```

```

char* token1 = strtok(content, ",");
char* token2 = strtok(NULL, ",");
char* token3 = strtok(NULL, ",");

if (token1 && token2 && token3) {
    Email new_email;
    strncpy(new_email.category, token1, sizeof(new_email.category) - 1);
    trim_whitespace(new_email.category);

    strncpy(new_email.subject, token2, sizeof(new_email.subject) - 1);
    trim_whitespace(new_email.subject);

    strncpy(new_email.date, token3, sizeof(new_email.date) - 1);
    trim_whitespace(new_email.date);

    new_email.category_priority = get_category_priority(new_email.category);
    new_email.date_val = parse_date(new_email.date);

    insert_email(heap, new_email);
}
}
else if (strcmp(line, "NEXT") == 0) {
    if (heap->size == 0) {
        printf("No unread emails.\n");
    } else {
        Email next_email = peek_max(heap);
        printf("Sender: %s\n", next_email.category);
        printf("Subject: %s\n", next_email.subject);
        printf("Date: %s\n", next_email.date);
    }
}
else if (strcmp(line, "READ") == 0) {
    if (heap->size > 0) {
        extract_max(heap);
    }
}
else if (strcmp(line, "COUNT") == 0) {
    printf("%d\n", heap->size);
}
}

```

Copilot:

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_SUBJECT 256
#define MAX_CATEGORY 64

// -----
// Email Structure
// -----
typedef struct {
    char category[MAX_CATEGORY];
    char subject[MAX_SUBJECT];
    int month, day, year;
    int priority; // computed priority
} Email;

// -----
// MaxHeap Structure
// -----
typedef struct {
    Email heap[MAX_EMAILS];
    int size;
} MaxHeap;

// -----
// Priority Mapping
// -----
int categoryPriority(const char *cat) {
    if (strcmp(cat, "Boss") == 0) return 5;
    if (strcmp(cat, "Subordinate") == 0) return 4;
    if (strcmp(cat, "Peer") == 0) return 3;
    if (strcmp(cat, "ImportantPerson") == 0) return 2;
    if (strcmp(cat, "OtherPerson") == 0) return 1;
    return 0;
}

// -----
// Compare Emails by Priority
// -----
int compareEmails(Email a, Email b) {
    if (a.priority != b.priority)
        return a.priority - b.priority;

```

```

// If same category priority, newest date first
if (a.year != b.year)
    return a.year - b.year;
if (a.month != b.month)
    return a.month - b.month;
return a.day - b.day;
}

// -----
// Heap Helper Functions
// -----
void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapifyUp(MaxHeap *h, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compareEmails(h->heap[index], h->heap[parent]) > 0) {
            swap(&h->heap[index], &h->heap[parent]);
            index = parent;
        } else break;
    }
}

void heapifyDown(MaxHeap *h, int index) {
    int left, right, largest;

    while (1) {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;

        if (left < h->size && compareEmails(h->heap[left], h->heap[largest]) > 0)
            largest = left;
        if (right < h->size && compareEmails(h->heap[right], h->heap[largest]) > 0)
            largest = right;

        if (largest != index) {
            swap(&h->heap[index], &h->heap[largest]);
            index = largest;
        } else break;
    }
}

```

```

    }
}

// -----
// Heap Operations
// -----
void insertEmail(MaxHeap *h, Email e) {
    if (h->size >= MAX_EMAILS) {
        printf("Heap full, cannot insert.\n");
        return;
    }
    h->heap[h->size] = e;
    heapifyUp(h, h->size);
    h->size++;
}

Email* peekEmail(MaxHeap *h) {
    if (h->size == 0) return NULL;
    return &h->heap[0];
}

void popEmail(MaxHeap *h) {
    if (h->size == 0) return;
    h->heap[0] = h->heap[h->size - 1];
    h->size--;
    heapifyDown(h, 0);
}

// -----
// Parsing Email File Format
// -----
Email parseEmail(FILE *fp) {
    Email e;
    char line[512];

    fgets(line, sizeof(line), fp);
    sscanf(line, "EMAIL %s", e.category);

    fgets(line, sizeof(line), fp);
    line[strcspn(line, "\n")] = 0;
    strcpy(e.subject, line);

    fgets(line, sizeof(line), fp);
    sscanf(line, "%d-%d-%d", &e.month, &e.day, &e.year);
}

```

```

    e.priority = categoryPriority(e.category);
    return e;
}

// -----
// Main Program
// -----
int main() {
    MaxHeap inbox = { .size = 0 };
    char command[64];

    while (scanf("%s", command) != EOF) {

        if (strcmp(command, "EMAIL") == 0) {
            // Put back "EMAIL" line into parsing buffer
            ungetc('\n', stdin);
            ungetc('L', stdin);
            ungetc('I', stdin);
            ungetc('A', stdin);
            ungetc('M', stdin);
            ungetc('E', stdin);

            Email e = parseEmail(stdin);
            insertEmail(&inbox, e);
        }

        else if (strcmp(command, "NEXT") == 0) {
            Email *e = peekEmail(&inbox);
            if (!e) {
                printf("No emails.\n");
            } else {
                printf("Sender: %s\n", e->category);
                printf("Subject: %s\n", e->subject);
                printf("Date: %02d-%02d-%04d\n", e->month, e->day, e->year);
            }
        }

        else if (strcmp(command, "READ") == 0) {
            if (inbox.size == 0) {
                // Nothing to delete
            } else {
                popEmail(&inbox);
            }
        }
    }
}

```

```

    }

    else if (strcmp(command, "COUNT") == 0) {
        printf("%d\n", inbox.size);
    }
}

return 0;
}

```

Compared: the Gemini version basically successfully runs the max-heap function, including insertEmail, peekMax, and extractMax

They use sift_up() and sift_down() to maintain the max-heap, and it also meets the requirement of separating the sender, category, subject, and date into three parts like EMAIL Peer, Can you help me on this?,12-01-2024"

The output has a small problem: the count outputs only 6 instead of "There are 6 emails to read." And it doesn't have Next email:" but that's not a big issue

For the copilot, the logic of the max heap is basically correct it uses heapify-up and heapify-down, and he got the compareEmails logic

But it has an error in handling the input

EMAIL <sender category>,<subject line>,<date> and when i run it in

The output is:

Sender:

Subject: Peer, can you help me with this?,12-01-2024

Date: 00-00-0000

The date is not the correct one and it should be in the same line

The output form has a similar problem with Gemini

Gemini gets the points

#For the execution time

The time complexity of both of them

insert functions is $O(\log n)$

next $-O(1)$

Read $-O(\log n)$

Which means their mexheap doesn't have too much difference

for the space complexity:

The Gemini uses the allocated array to store the emails. Its capacity can increase when the heap becomes full. Therefore, storing n emails requires $O(n)$ space.

The Copilot program uses a fixed-size array with a max of 1000 emails. It also requires $O(n)$ space for the emails, but it has a fixed maximum capacity.

Both of their space complexities are $O(n)$

maintainability

Gemini's code is organized into several clear functions, with separate functions for priority calculation, date parsing, email comparison, heap operations, and command processing. This modular structure makes the code easier to understand, debug, and modify. And it didn't have a max email limit, so it's very flexible

Coilpot also clearly used named functions, but his heap-related functions are easy to understand. But its input processing section is more complicated because parseemail does not follow the requirements of online and uses multiple ungetc() calls, so it makes the input processing part harder to understand, and some of the variable names are hard to understand

Both of them have the problem of having too few comments, so whatever you choose, you have to add lots of comments to the program

So Gemini functions are organized and more clearly in one place, making the whole program easy to read

But its input processing design is hard to read; you have to read through the whole function to identify the function it uses if you want to add more functions to this program, Gemini is easier Gemini got a point

choice

so I chose Gemini because it meets more requirements of the program and it has a clear structure

improvement

#correctness

The original Gemini code had two output problems: the count only printed the number of emails, and the next command did not print the next team. I changed both outputs to get the correct sample output

I fixed the trim with space function so that it operates correctly and deals with the space, and removed

I added the `\0` after `strncpy()` to make sure the copied strings are properly terminated, and I added checks for `malloc` and `realloc` so they can handle memory allocation errors

Execution time

I don't need to improve too much because it already use the maxheap so it's can be optimizing a lot in terms of time complexity

I kept the dynamic-array MaxHeap from the Gemini code. The program uses $O(n)$ space for n unread emails. The initial capacity is 10, and the array doubles when it becomes full. This lets the program store more emails without a fixed maximum size. No additional data structure was needed, so the overall space complexity remains $O(n)$.

Maintainability I kept the original Gemini function structure because the program is divided into clear functions for priority calculation, date conversion, email comparison, heap operations, and command processing. I added more comments to make the code easier to understand. I also improved memory allocation and string handling, making the program safer and easier to modify.

```
#include <stdio.h> // Include functions for input and output.
#include <stdlib.h> // Include functions for memory allocation.
#include <string.h> // Include functions for string operations.
#include <ctype.h> // Include functions for checking spaces.

#define INITIAL_CAPACITY 10 // Set the starting heap capacity.

// Email structure
typedef struct {
    char category[50]; // Store the sender category.
    int category_priority; // Store the priority of the sender.
    char subject[256]; // Store the email subject.
    char date[11]; // Store the email date.
    long date_val; // Store the date as a number.
} Email;

// MaxHeap structure (List-based / Dynamic Array)
typedef struct {
    Email* data; // Store the emails in a dynamic array.
    int size; // Store the current number of emails.
    int capacity; // Store the current array capacity.
} MaxHeap;

// Function declarations
MaxHeap* create_heap(); // Create a new heap.
void free_heap(MaxHeap* heap); // Free the heap memory.
int get_category_priority(const char* cat); // Get the priority of a category.
long parse_date(const char* date_str); // Convert a date into a number.
int compare_emails(Email a, Email b); // Compare two emails.
void swap(Email* a, Email* b); // Swap two emails.
void sift_up(MaxHeap* heap, int index); // Move an email up the heap.
void sift_down(MaxHeap* heap, int index); // Move an email down the heap.
void insert_email(MaxHeap* heap, Email email); // Insert an email into the heap.
Email extract_max(MaxHeap* heap); // Remove the highest-priority email.
Email peek_max(MaxHeap* heap); // Get the highest-priority email.
void trim_whitespace(char* str); // Remove extra spaces.
void process_command(MaxHeap* heap, char* line); // Process an input command.

// Main function
```

```

int main(int argc, char* argv[]) {
    FILE* file = stdin; // Use standard input by default.

    // Check if an input file was provided.
    if (argc > 1) {
        file = fopen(argv[1], "r"); // Open the input file.
        if (!file) { // Check if the file opened successfully.
            perror("Error opening file"); // Print an error message.
            return 1; // Stop the program if the file cannot be opened.
        }
    }

    MaxHeap* heap = create_heap(); // Create an empty heap.
    char line[512]; // Create a buffer for each input line.

    // Read each line from the input.
    while (fgets(line, sizeof(line), file)) {
        line[strcspn(line, "\r\n")] = 0; // Remove the newline character.
        if (strlen(line) == 0) continue; // Skip empty lines.

        process_command(heap, line); // Process the current command.
    }

    // Close the file if an input file was used.
    if (file != stdin) {
        fclose(file); // Close the input file.
    }

    free_heap(heap); // Free the heap memory.
    return 0; // End the program successfully.
}

// Helper: Convert category to numerical priority
int get_category_priority(const char* cat) {
    if (strcmp(cat, "Boss") == 0) return 5; // Give Boss the highest priority.
    if (strcmp(cat, "Subordinate") == 0) return 4; // Give Subordinate priority 4.
    if (strcmp(cat, "Peer") == 0) return 3; // Give Peer priority 3.
    if (strcmp(cat, "ImportantPerson") == 0) return 2; // Give ImportantPerson priority 2.
    if (strcmp(cat, "OtherPerson") == 0) return 1; // Give OtherPerson the lowest priority.
    return 0; // Return 0 for an unknown category.
}

// Helper: Convert MM-DD-YYYY to a comparable integer YYYYMMDD
long parse_date(const char* date_str) {

```

```

int mm, dd, yyyy; // Store the month, day, and year.

// Read the month, day, and year from the date.
if (sscanf(date_str, "%d-%d-%d", &mm, &dd, &yyyy) == 3) {
    return (long)yyyy * 10000 + mm * 100 + dd; // Convert the date to YYYYMMDD.
}

return 0; // Return 0 if the date cannot be parsed.
}

// Compare two emails: returns 1 if 'a' has higher priority than 'b', 0 otherwise.
// Higher category priority comes first. For the same category, newer date comes first.
int compare_emails(Email a, Email b) {
    // Check the sender priorities first.
    if (a.category_priority != b.category_priority) {
        return a.category_priority > b.category_priority; // Return 1 if a has higher priority.
    }

    // Newer date means larger date value.
    return a.date_val > b.date_val; // Return 1 if a is newer.
}

// Create heap
MaxHeap* create_heap() {
    MaxHeap* heap = (MaxHeap*)malloc(sizeof(MaxHeap)); // Allocate memory for the heap.

    // Check if the heap memory was allocated.
    if (heap == NULL) {
        printf("Memory allocation failed.\n"); // Print an error message.
        exit(1); // Stop the program.
    }

    heap->capacity = INITIAL_CAPACITY; // Set the starting capacity.
    heap->size = 0; // Start with zero emails.
    heap->data = (Email*)malloc(heap->capacity * sizeof(Email)); // Allocate space for emails.

    // Check if the email array was allocated.
    if (heap->data == NULL) {
        printf("Memory allocation failed.\n"); // Print an error message.
        free(heap); // Free the heap structure.
        exit(1); // Stop the program.
    }

    return heap; // Return the new heap.
}

```

```
}
```

```
// Free heap memory
```

```
void free_heap(MaxHeap* heap) {  
    if (heap) { // Check if the heap exists.  
        free(heap->data); // Free the email array.  
        free(heap); // Free the heap structure.  
    }  
}
```

```
// Swap two emails
```

```
void swap(Email* a, Email* b) {  
    Email temp = *a; // Save the first email.  
    *a = *b; // Put the second email into the first position.  
    *b = temp; // Put the saved email into the second position.  
}
```

```
// Sift Up operation
```

```
void sift_up(MaxHeap* heap, int index) {  
    // Continue until the email reaches the root.  
    while (index > 0) {  
        int parent = (index - 1) / 2; // Find the parent index.  
  
        // Check if the current email has higher priority.  
        if (compare_emails(heap->data[index], heap->data[parent])) {  
            swap(&heap->data[index], &heap->data[parent]); // Swap the email with its parent.  
            index = parent; // Continue from the parent's position.  
        } else {  
            break; // Stop when the heap order is correct.  
        }  
    }  
}
```

```
// Sift Down operation
```

```
void sift_down(MaxHeap* heap, int index) {  
    int max_index = index; // Start with the current position.  
    int left = 2 * index + 1; // Find the left child.  
    int right = 2 * index + 2; // Find the right child.  
  
    // Check if the left child has higher priority.  
    if (left < heap->size && compare_emails(heap->data[left], heap->data[max_index])) {  
        max_index = left; // Set the left child as the highest priority.  
    }  
}
```

```

// Check if the right child has higher priority.
if (right < heap->size && compare_emails(heap->data[right], heap->data[max_index])) {
    max_index = right; // Set the right child as the highest priority.
}

// Check if the current position needs to be changed.
if (max_index != index) {
    swap(&heap->data[index], &heap->data[max_index]); // Swap with the higher-priority child.
    sift_down(heap, max_index); // Continue fixing the heap.
}
}

// Insert email into MaxHeap
void insert_email(MaxHeap* heap, Email email) {
    // Check if the heap needs more space.
    if (heap->size >= heap->capacity) {
        heap->capacity *= 2; // Double the heap capacity.

        // Create a temporary pointer for the resized array.
        Email* temp = (Email*)realloc(heap->data, heap->capacity * sizeof(Email));

        // Check if the memory was resized successfully.
        if (temp == NULL) {
            printf("Memory allocation failed.\n"); // Print an error message.
            free(heap->data); // Free the old email array.
            free(heap); // Free the heap.
            exit(1); // Stop the program.
        }

        heap->data = temp; // Use the resized array.
    }

    heap->data[heap->size] = email; // Add the new email at the end.
    sift_up(heap, heap->size); // Move the email to the correct position.
    heap->size++; // Increase the number of emails.
}

// Extract max email (READ action)
Email extract_max(MaxHeap* heap) {
    Email empty = {"", 0, "", "", 0}; // Create an empty email.

    // Check if there are no emails.
    if (heap->size == 0) return empty; // Return the empty email.

```

```

    Email root = heap->data[0]; // Save the highest-priority email.
    heap->data[0] = heap->data[heap->size - 1]; // Move the last email to the root.
    heap->size--; // Decrease the number of emails.
    sift_down(heap, 0); // Restore the heap order.
    return root; // Return the removed email.
}

// Peek max email (NEXT action)
Email peek_max(MaxHeap* heap) {
    Email empty = {"", 0, "", "", 0}; // Create an empty email.

    // Check if there are no emails.
    if (heap->size == 0) return empty; // Return the empty email.

    return heap->data[0]; // Return the highest-priority email.
}

// Trim leading and trailing whitespace
void trim_whitespace(char* str) {
    char* start = str; // Start at the beginning of the string.
    char* end; // Create a pointer for the end of the string.

    while (isspace((unsigned char)*start)) start++; // Skip spaces at the beginning.

    // Check if the string is empty.
    if (*start == 0) {
        str[0] = '\0'; // Make the string empty.
        return; // Stop the function.
    }

    end = start + strlen(start) - 1; // Find the last character.

    while (end > start && isspace((unsigned char)*end)) end--; // Remove spaces at the end.

    *(end + 1) = '\0'; // Add the string terminator.

    // Move the trimmed string to the beginning if needed.
    if (start != str) {
        memmove(str, start, strlen(start) + 1); // Move the string to the original location.
    }
}

// Process individual commands from the test file/input
void process_command(MaxHeap* heap, char* line) {

```

```

// Check if the command is EMAIL.
if (strncmp(line, "EMAIL", 5) == 0) {
    // Format: EMAIL <sender category>, <subject line>, <date>
    char* content = line + 5; // Skip the word EMAIL.

    while (*content && isspace((unsigned char)*content)) content++; // Skip spaces after
    EMAIL.

    char* token1 = strtok(content, ","); // Get the sender category.
    char* token2 = strtok(NULL, ","); // Get the subject.
    char* token3 = strtok(NULL, ","); // Get the date.

    // Check that all three email fields were found.
    if (token1 && token2 && token3) {
        Email new_email; // Create a new email.

        strncpy(new_email.category, token1, sizeof(new_email.category) - 1); // Copy the
category.
        new_email.category[sizeof(new_email.category) - 1] = '\0'; // End the category string.
        trim_whitespace(new_email.category); // Remove extra spaces.

        strncpy(new_email.subject, token2, sizeof(new_email.subject) - 1); // Copy the subject.
        new_email.subject[sizeof(new_email.subject) - 1] = '\0'; // End the subject string.
        trim_whitespace(new_email.subject); // Remove extra spaces.

        strncpy(new_email.date, token3, sizeof(new_email.date) - 1); // Copy the date.
        new_email.date[sizeof(new_email.date) - 1] = '\0'; // End the date string.
        trim_whitespace(new_email.date); // Remove extra spaces.

        new_email.category_priority = get_category_priority(new_email.category); // Set the
email priority.
        new_email.date_val = parse_date(new_email.date); // Convert the date to a number.

        insert_email(heap, new_email); // Add the email to the heap.
    }
}
else if (strcmp(line, "NEXT") == 0) {
    // Check if there are no unread emails.
    if (heap->size == 0) {
        printf("No unread emails.\n"); // Display the empty heap message.
    } else {
        Email next_email = peek_max(heap); // Get the highest-priority email.
        printf("Next email:\n"); // Display the NEXT heading.
        printf("Sender: %s\n", next_email.category); // Display the sender.
    }
}

```

```
        printf("Subject: %s\n", next_email.subject); // Display the subject.
        printf("Date: %s\n", next_email.date); // Display the date.
    }
}
else if (strcmp(line, "READ") == 0) {
    // Check if there is an email to read.
    if (heap->size > 0) {
        extract_max(heap); // Remove the highest-priority email.
    }
}
else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heap->size); // Display the number of unread
emails.
}
}
```